

GRAFIKA KOMPUTEROWA I KOMUNIKACJA Z KOMPUTEREM

Wykład 4

Andrzej Śluzek

pokój: 3/64

email: andrzej_sluzek@sggw.edu.pl



Tematyka wykładów i zajęć laboratoryjnych

(oficjalny sylabus, kolejność i tematyka może być nieco inna)

1. Wstęp. Urządzenia graficzne wejścia-wyjścia.
2. Podstawowe pojęcia i definicje dotyczące obrazów cyfrowych (w technice rastrowej i wektorowej) oraz modeli kolorów.
3. Punktowe, lokalne i globalne metody przekształcania obrazów.
4. Transformacje geometryczne obiektów graficznych i **wypełnianie figur**.
5. **Eliminacja elementów zasłoniętych**, metody kreślenia krzywych.
6. Modele oświetlenia i cieniowania.
7. Przekształcenia morfologiczne i ich zastosowania.
8. Transformaty obrazów i sygnałów dźwiękowych (FFT, Hough, Radon). Wybrane metody filtracji danych jedno- i dwuwymiarowych.
9. Wybrane modele interakcji człowiek-komputer. Zasady projektowania i analizy interfejsów komputerowych.
10. Interfejsy dla osób niepełnosprawnych. Interakcja człowiek-komputer dla komputerów przyszłości.

Wypełnianie konturów

Modele graficzne często używają konturów definiujących kształt obiektów, które powinny być przedstawione na obrazie.

Realistyczne wygenerowanie takiego obrazu wymaga *wypełnienia konturów*, tzn. nadanie poszczególnym obiektom zadanego koloru (lub poziomu jasności).

Najpowszechniej stosowany i zarazem bardzo prosty (konceptyjnie i obliczeniowo) jest algorytm *flood-fill*.

W swojej teoretycznej definicji ma on postać *rekurencyjną*, która jednak nie jest stosowana w praktyce (powoduje szybkie przepełnienie stosu).

Zamiast tego stosuje się implementację kolejkową wykorzystującą listę tzw. *aktywnych pixeli*.

Długość tej listy nie może nigdy przekroczyć liczby pixeli w obrazie (a więc nie ma praktycznej możliwości przepełnienia), a w dodatku algorytm w tej postaci jest znacznie szybszy (i łatwo jest go implementować sprzętowo, np. w postaci wyspecjalizowanych układów scalonych).

Oba warianty będą pokazane bardziej szczegółowo na następnych slajdach.

Wypełnianie konturów

Algorytm flood-fill

Może być używany do następujących zadań (a również innych celów) :

- Wypełnianie konturów.
- Zmiany kolorów wybranych obszarów obrazu (przemalowanie).
- Znajdowanie w obrazie obszarów o jednolitych, albo zbliżonych, kolorach/jasnościach (i zazwyczaj zamalowywanie tych obszarów unikalnymi kolorami/jasnościami).

W wersji wypełniania konturów zakładamy, że dany jest:

1. Obraz z zaznaczonymi liniami konturowymi ograniczającymi obszary obrazu (brzegi obrazu też są traktowane jako ograniczające kontury).
2. Współrzędne startowego pixela wewnątrz konturu, który chcemy wypełnić.
3. Kolor/jasność wypełnienia.
4. Dodatkowo zwykle definiuje się kolor/jasność tła, tzn. „pustych” części obrazu (dla uniknięcia błędów ze startowymi pixelami).

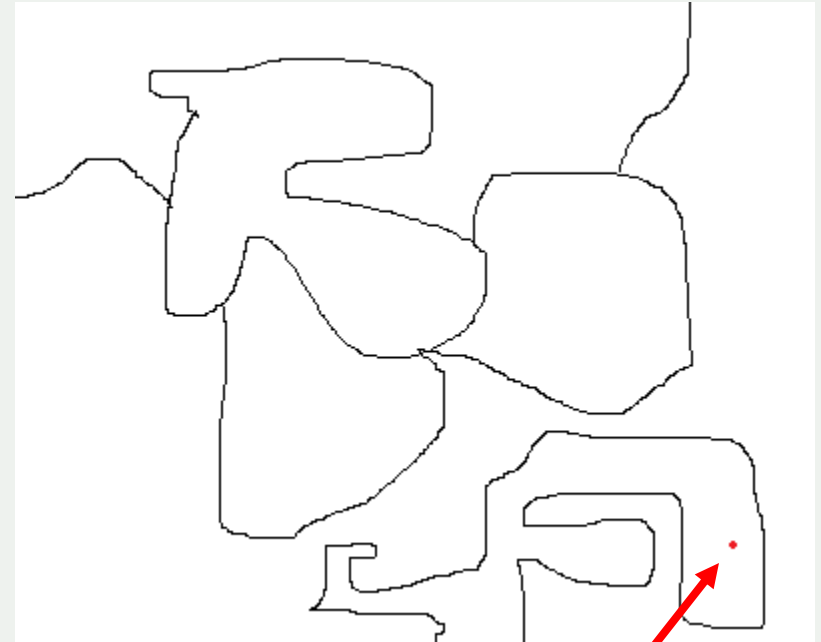
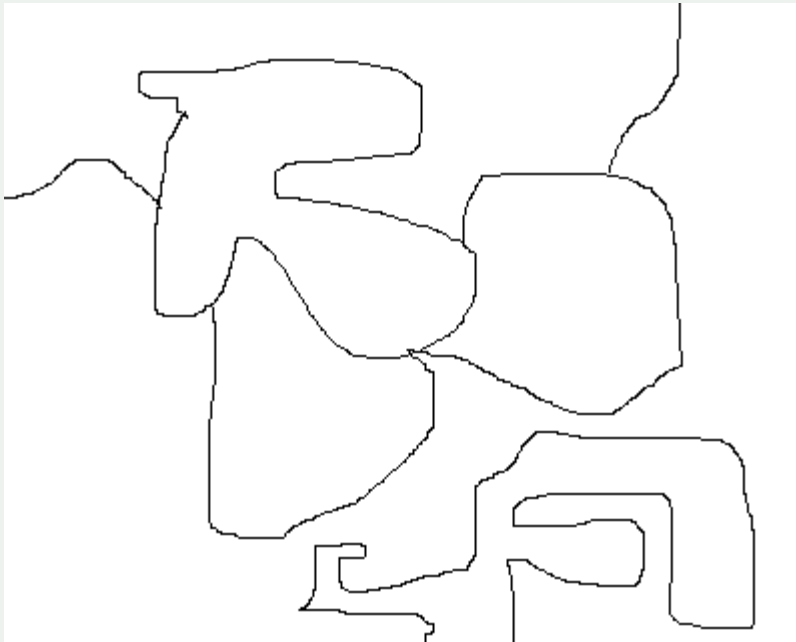
Wypełnianie konturów

Algorytm flood-fill

Pixel początkowy (360, 272).

Kolor tła: **biały**.

Kolor wypełnienia: **czzerwony**



Wypełnianie konturów

Algorytm flood-fill

Zmiana koloru w (jednolitym) obszarze na nowy kolor.

Kolorowanie nowym kolorem kolejnych pixeli trwa tak długo, jak długo mają one tę samą jasność (kolor) co pixel startowy. *De facto* jest to ten sam problem, co na poprzednim slajdzie. Jedyna różnica, to „kolor tła”, który jest oryginalnym kolorem danego obszaru.



Wypełnianie konturów

Algorytm flood-fill

Pseudo-kod algorytmu (wersja rekurencyjna)

```
Flood-fill (pixel, kolor_tła, kolor_wypełnienia)  
1. If kolor pixela nie jest kolorem tła, return.  
2. else zmień kolor pixela na kolor_wypełnienia.  
3. Flood-fill (lewy sąsiad pixela, kolor_tła,  
               kolor_wypełnienia)  
4. Flood-fill (prawy sąsiad pixela, kolor_tła,  
               kolor_wypełnienia)  
5. Flood-fill (dolny sąsiad pixel, kolor_tła,  
               kolor_wypełnienia)  
6. Flood-fill (górny sąsiad pixel, kolor_tła,  
               kolor_wypełnienia)  
7. Return.
```

Wypełnianie konturów

Algorytm flood-fill

Pseudo-kod algorytmu (wersja kolejkowa)

Flood-fill (*pixel_pocz*, *kolor_tła*, *kolor_wypełnienia*):

1. **If** kolor *pixela_pocz* nie jest *kolorem tła*, **return**.
2. Utwórz pustą listę (kolejkę) *Q*.
3. Dodaj *pixel_pocz* na koniec kolejki *Q*.
4. Jeśli kolejka *Q* nie jest pusta, pobierz *pixel* z początku kolejki *Q*:

Zmień kolor *pixela* na *kolor_wypełnienia* i usuń *pixel* z kolejki *Q*.

If prawy sąsiad *pixela* (*pxr*) ma *kolor_tła*, dodaj *pxr* na koniec kolejki *Q*.

If lewy sąsiad *pixela* (*pxl*) ma *kolor_tła*, dodaj *pxl* na koniec kolejki *Q*.

If dolny sąsiad *pixela* (*pxd*) ma *kolor_tła*, dodaj *pxd* na koniec kolejki *Q*.

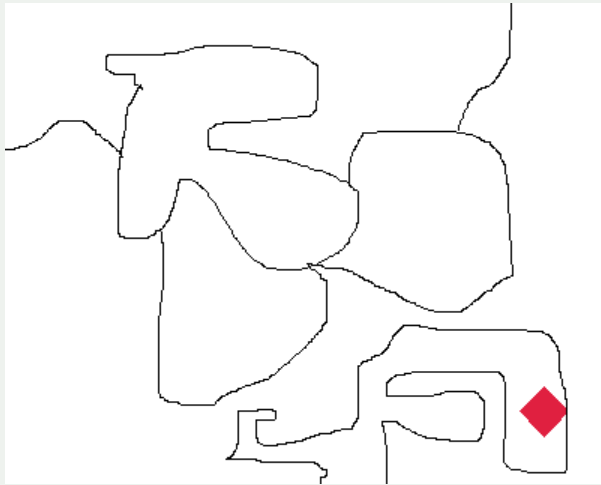
If górny sąsiad *pixela* (*pxu*) ma *kolor_tła*, dodaj *pxu* na koniec kolejki *Q*.

5. Kontynuuj pętlę (4) aż do opróżnienia kolejki *Q*.
6. **Return**.

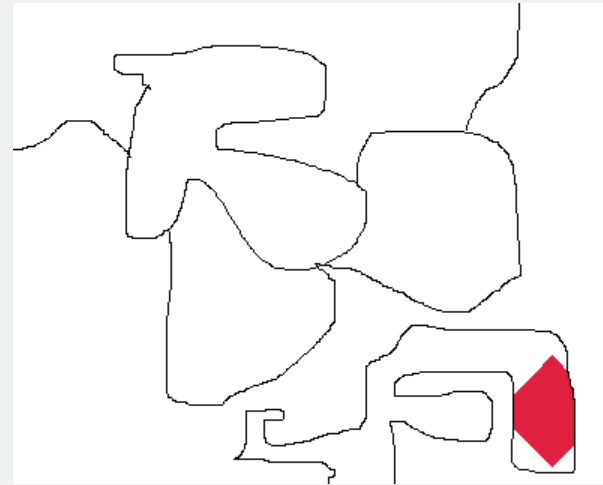
Wypełnianie konturów

Algorytm flood-fill

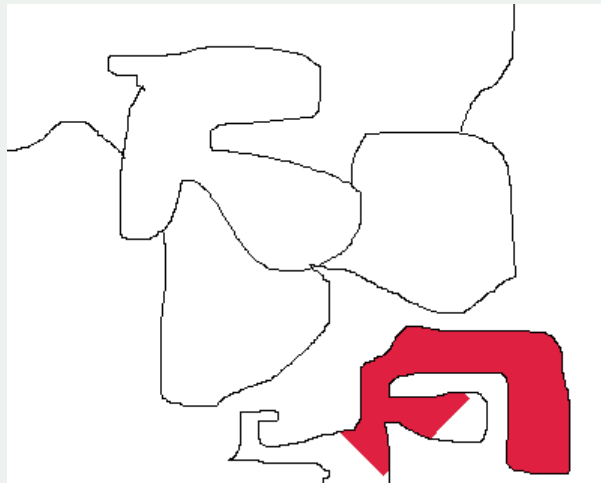
Przykładowe wyniki:



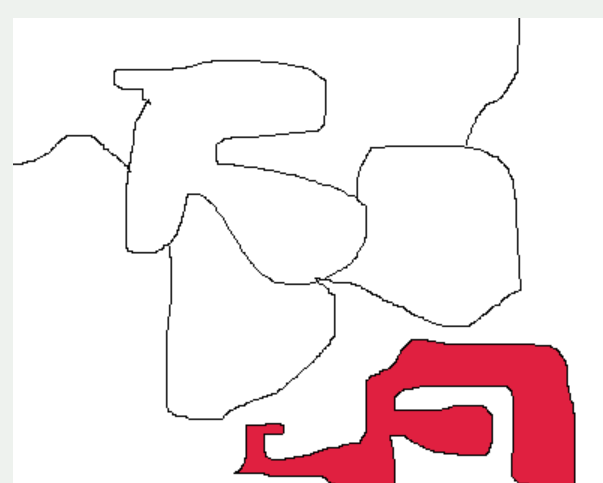
500 iteracji



2000 iteracji



8000 iteracji



wynik
końcowy

Wypełnianie konturów

Algorytm flood-fill

Praktycznie ten sam algorytm może być użyty do zamalowywania już istniejących obszarów nowymi kolorami lub jasnościami (Slajd 6).

Wypełnianie konturów

Algorytm flood-fill

Przykładowe wyniki:



Wypełnianie konturów/segmentacja

Algorytmu *flood-fill* można użyć też do podziału obrazów monochromatycznych lub kolorowych na (w miarę) jednolite obszary, czyli do *segmentacji* obrazu.

Pseudo-kod algorytmu

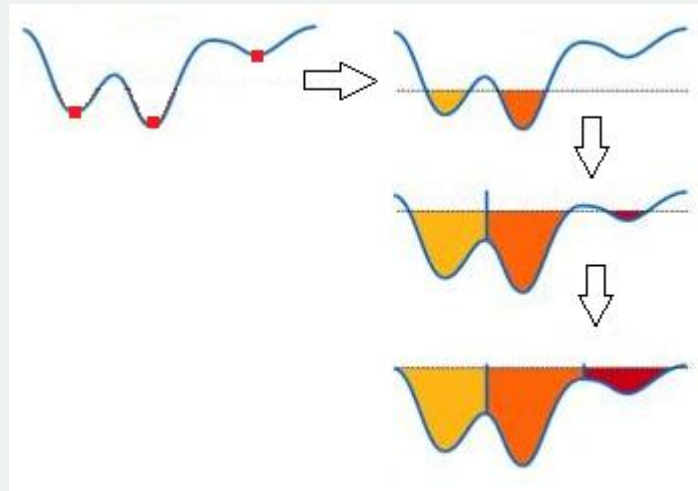
Flood-fill (*pixel_pocz*, *kolor_pocz*, *kolor_wypełnienia*):

1. **If** kolor *pixela_pocz* nie jest podobny do *koloru_pocz*, **return**.
2. Utwórz pustą listę (kolejkę) *Q*.
3. Dodaj *pixel_pocz* na koniec kolejki *Q*.
4. Dopóki kolejka *Q* nie jest pusta, pobierz *pixel* z początku kolejki *Q*:
Zmień kolor *pixela* na *kolor_wypełnienia* i usuń *pixel* z kolejki *Q*.
If prawy sąsiad *pixela* (*pxr*) jest podobny do *koloru_pocz*, dodaj *pxr* na koniec kolejki *Q*.
If lewy sąsiad *pixela* (*pxl*) jest podobny do *koloru_pocz*, dodaj *pxl* na koniec kolejki *Q*.
If dolny sąsiad *pixela* (*pxd*) jest podobny do *koloru_pocz*, dodaj *pxd* na koniec kolejki *Q*.
If górny sąsiad *pixela* (*pxu*) jest podobny do *koloru_pocz*, dodaj *pxu* na koniec kolejki *Q*.
5. Kontynuuj pętlę (4) aż do opróżnienia kolejki *Q*.
6. **Return**.

Wypełnianie konturów/segmentacja

Algorytm *watershed*

Algorytm ten jest alternatywną metodą podziału obrazów na (w miarę) jednolite obszary. Podstawową ideę (w wariancie jednowymiarowym) algorytmu *watershed* (czyli *działów wodnych*) ilustruje rysunek:



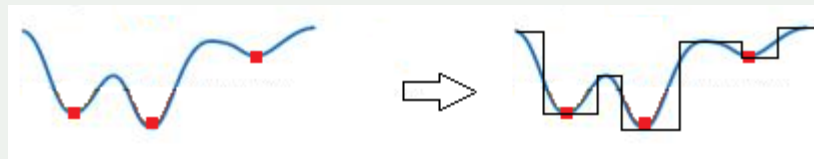
Ogólnie, algorytm startuje z lokalnych minimów funkcji jasności, które stają się „źródłami wody” zalewającej „teren”.

Granice między obszarami powstają wzdłuż linii, gdzie spotykają się wody wypływające z różnych źródeł.

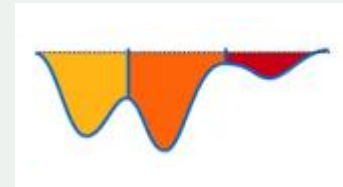
Wypełnianie konturów/segmentacja

Algorytm *watershed*

Intuicyjnie, wynik segmentacji obrazu (tzn. jego funkcji jasności) na w miarę jednolite obszary powinien być mniej więcej taki:



A jest taki:



„Coś jest nie tak” z algorytmem *watershed*.

Na następnym slajdzie widać, że wyniki tego algorytmu na obrazie monochromatycznym faktycznie nie mają większego sensu (inny przykład będzie na laboratorium).

Wypełnianie konturów/segmentacja

Algorytm *watershed*



Segmentacja metodą
watershed

Wypełnianie konturów/segmentacja

Algorytm *watershed*

W rzeczywistości, algorytm *watershed* stosuje się do segmentacji obrazów wykorzystując obrazy gradientowe (konturowe) zamiast obrazów oryginalnych.

Co to jest *obraz gradientowy*?

Obraz gradientowy jest obrazem uzyskanym z wartości (bezwzględnych) gradientu funkcji jasności.

Duża jasność takiego obrazu oznacza to, że co najmniej jedna z pochodnych cząstkowych funkcji jasności $im(x,y)$ ma dużą wartość bezwzględną, a więc jest tam krawędź.

Ogólnie obrazy gradientowe powstają jako:

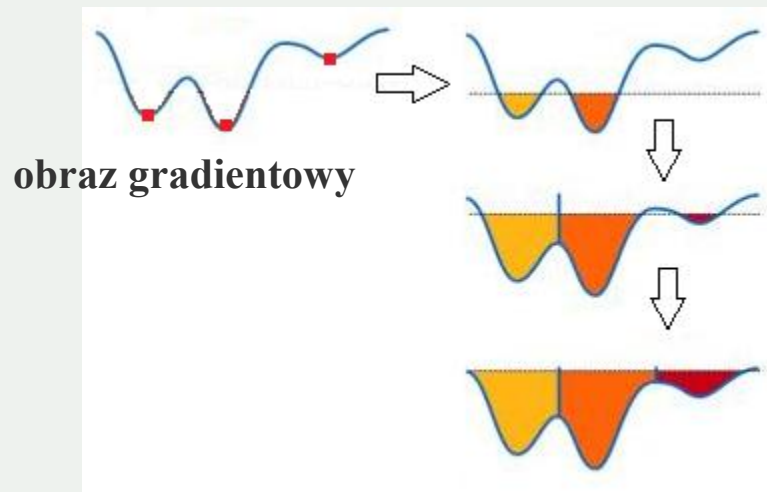
$$gi(x, y) = \left| \frac{\partial im(x, y)}{\partial x} \right| + \left| \frac{\partial im(x, y)}{\partial y} \right| \quad \text{albo} \quad gi(x, y) = \sqrt{\left(\frac{\partial im(x, y)}{\partial x} \right)^2 + \left(\frac{\partial im(x, y)}{\partial y} \right)^2}$$

Wypełnianie konturów/segmentacja

Algorytm *watershed*

Duże wartości w obrazach gradientowych oznaczają krawędzie, a więc granice między obszarami.

Dlatego też podział obrazu na (w miarę) jednolite obszary realizowany jest na obrazach gradientowych (a nie na obrazach oryginalnych)!

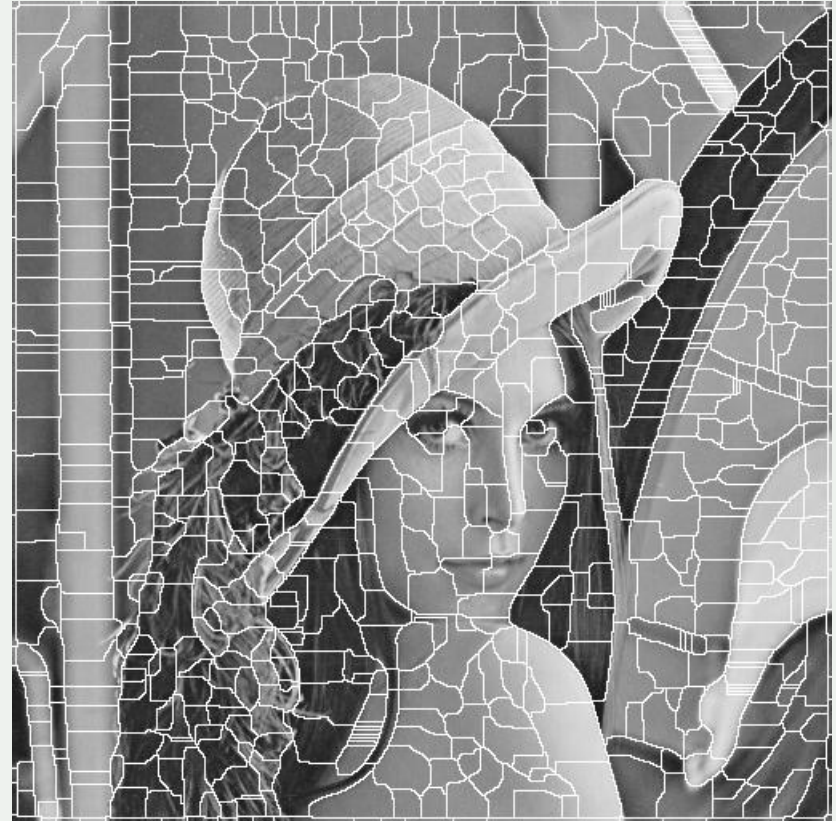


Wypełnianie konturów/segmentacja

Algorytm *watershed*



Obraz gradientowy.



Segmentacja metodą *watershed* na podstawie obrazu gradientowego

Wypełnianie konturów/segmentacja

Algorytm *watershed*

W kolejnej fazie, segmenty wyznaczone przez algorytm mogą być łączone w większe obszary.

Można to zrealizować używając różnych algorytmów (np. algorytmu *merge*, który jest drugim etapem metody *split-and-merge*).

Przykład:

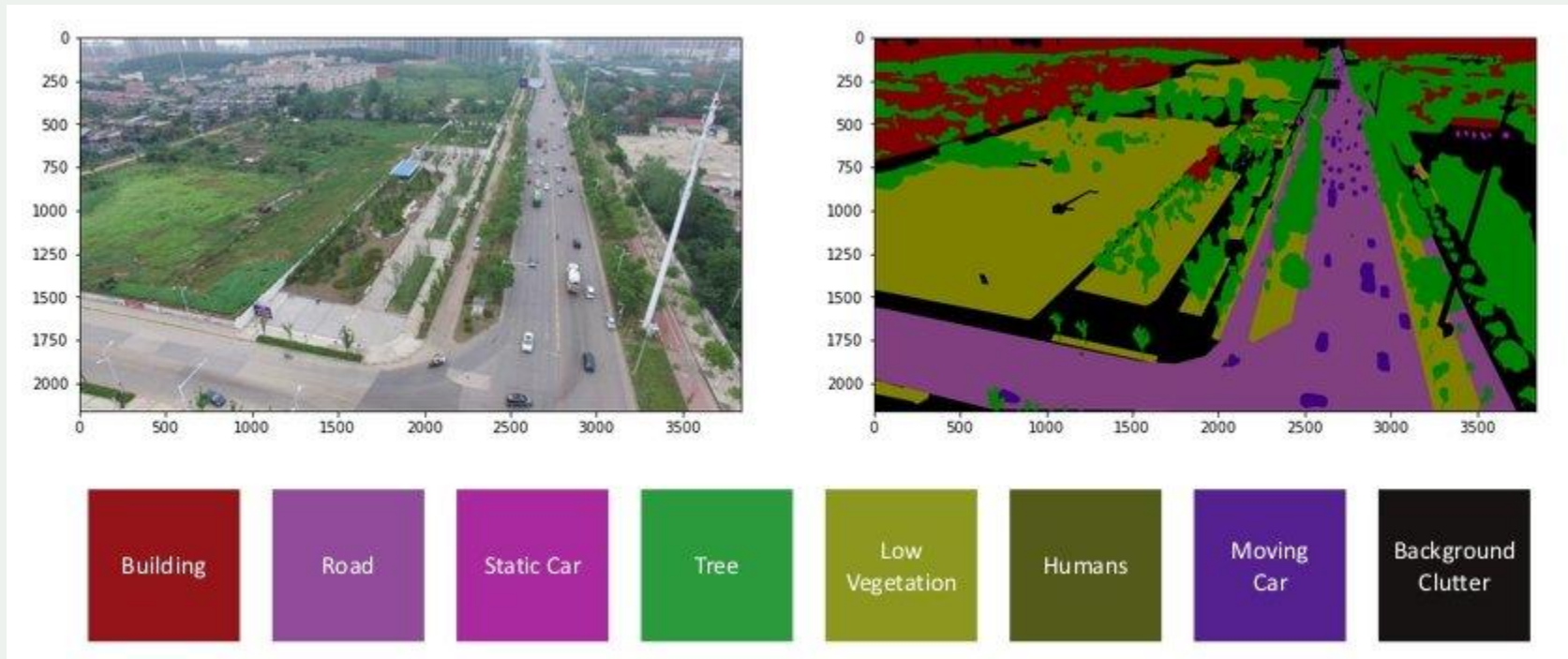


Wypełnianie konturów/segmentacja

Metody sztucznej inteligencji

Ostatnio coraz częściej wykorzystuje się wytrenowane sieci głębokie do segmentacji obrazów naturalnych na fragmenty, które nie muszą być w pełni jednolite wizualnie, ale reprezentują semantycznie jednorodne obiekty.

Wymaga to odpowiednio urozmaiconych danych uczących.



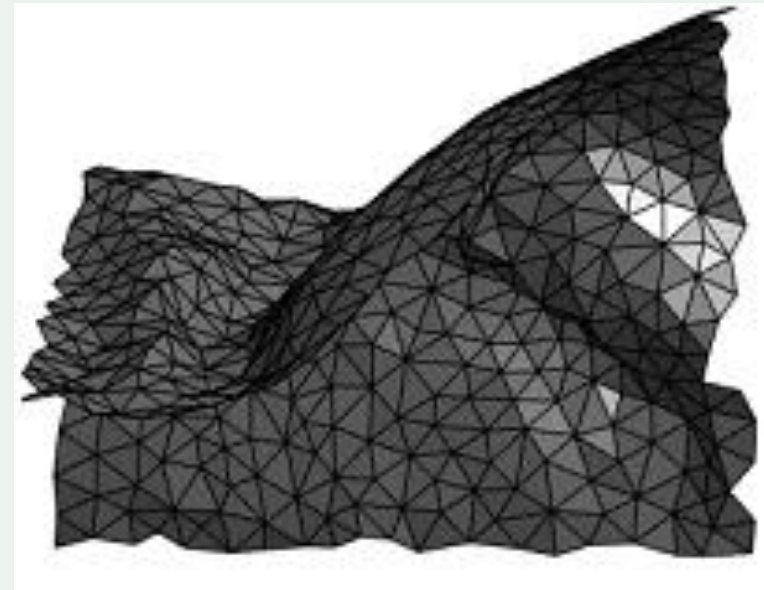
Eliminacja elementów zasłoniętych

Przy generacji obrazów przedstawiających obiekty 3D, każdy wielokąt powierzchni obiektu (powierzchnie zwykle są aproksymowane siatką np. trójkątów – patrz rysunek) może być zrzutowany na 2D obraz wg opisanych wcześniej transformacji z 3D do 2D.

Problemem jest jednak to, że nie wszystkie takie wielokąty są w pełni widoczne, a więc niektóre z nich nie powinny być w wyświetlone w całości.

1. Niektóre z nich fizycznie nie są zwrócone w stronę kamery.
2. Nawet jeśli są „właściwie” zwrócone, mogą być całkowicie lub częściowo przysłonięte przez inne wielokąty znajdujące się bliżej kamery.

Usuwanie (tzn. brak wyświetlania) takich zbędnych fragmentów określamy mianem *eliminacji elementów zasłoniętych*.

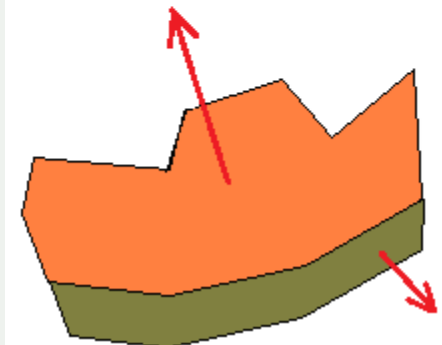
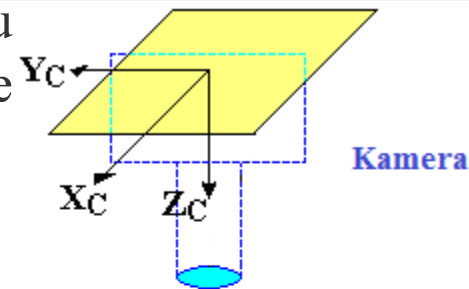
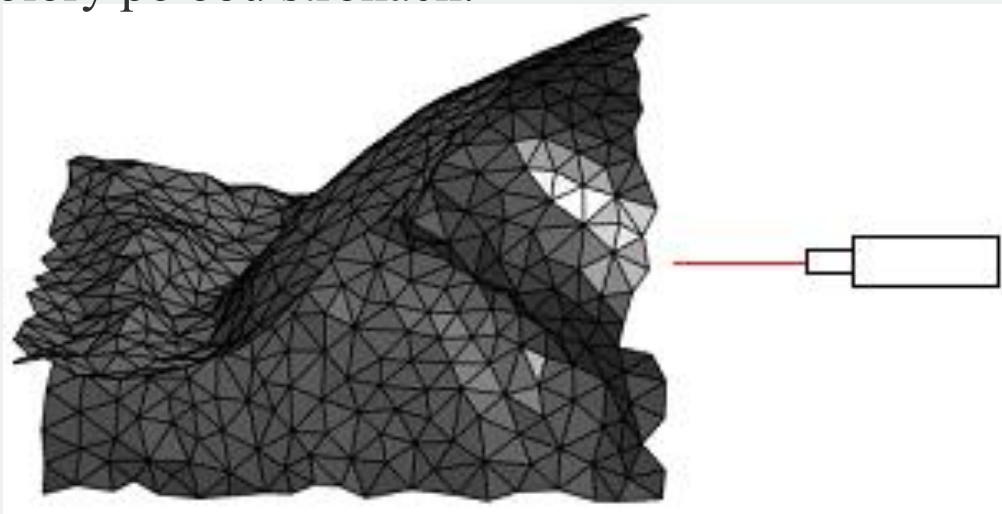
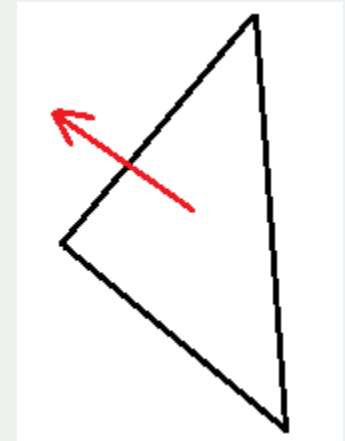


Eliminacja elementów zasłoniętych

Ad 1. Każdy wielokąt/trójkąt ma zdefiniowany kierunek wektora normalnego do jego widocznej powierzchni.

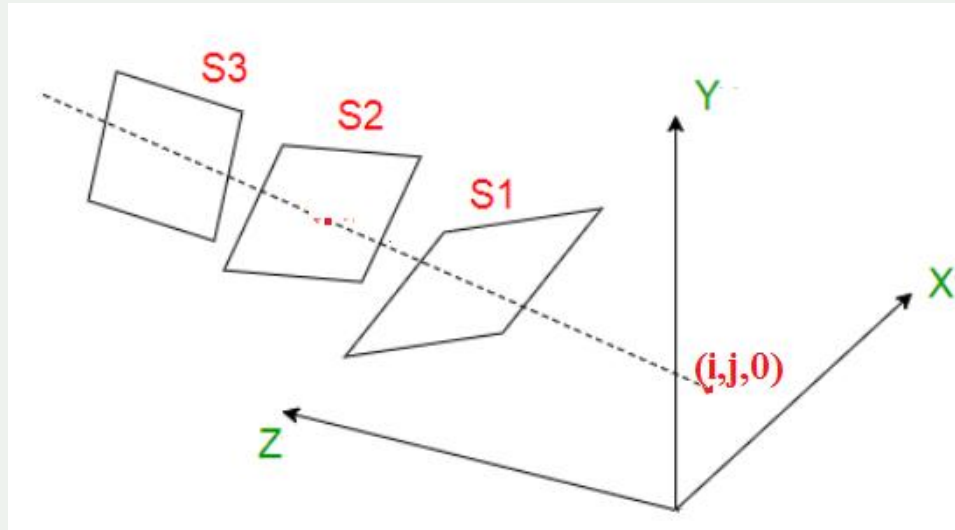
Jeśli wielokąt zwrócony jest widoczną powierzchnią w stronę kamery, można to zwykle (choć nie zawsze) stwierdzić sprawdzając, czy składowa **Z** jego wektora normalnego jest ujemna. Nadal może on być jednak częściowo lub całkowicie przesłonięty przez inne wielokąty znajdujące się bliżej kamery.

UWAGA: Podobny problem występuje w przypadku wielokątów, które są obustronnie widoczne, ale mają różne kolory po obu stronach.



Eliminacja elementów zasłoniętych

Ad 2. Wielokąty mogą się wzajemnie przesłaniać (całkowicie lub częściowo) .



Powszechnie stosowaną metodą wyznaczania, które nominalnie widoczne wielokąty (lub które ich fragmenty) powinny być wyświetlane jest tzw. algorytm *z-bufora* (z-buforowania).

Eliminacja elementów zasłoniętych

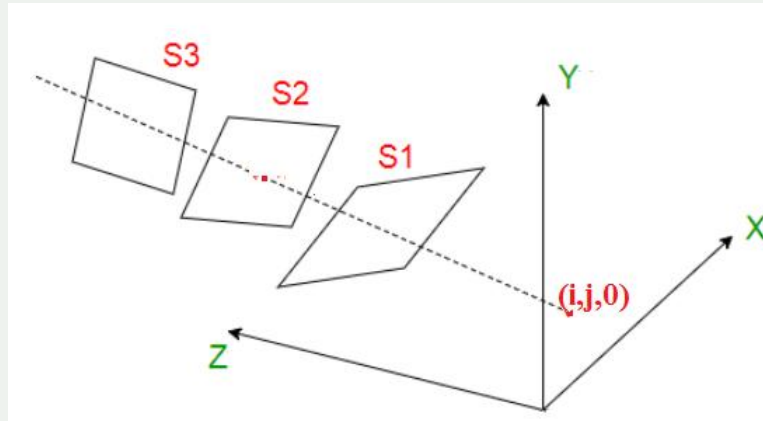
Algorytm z-bufora

Z-bufor zwany jest czasem *obrazem głębi*, a algorytm z-bufora pozwala znajdować kolor/jasność każdego pixela na obrazie niezależnie od innych pixeli (jest więc *operacją punktową*).

Algorytm używa dwóch kopii tworzonego obrazu:

- „Właściwy” obraz (np. $c(i,j)$).
- Obraz głębi, czyli z-bufor (np. $d(i,j)$).

Liczba obliczeń wykonywanych dla każdego pixela jest proporcjonalna do liczby wielokątów (trójkątów) w wykorzystywanym modelu, tzn. złożoność obliczeniowa metody jest $O(N \times K)$, gdzie jest N liczbą pixeli w obrazie, a K liczbą wielokątów.



Eliminacja elementów zasłoniętych

Algorytm z-bufora

Pseudo-kod (dla jednego punktu/pixela o współrzędnych (i, j))

Ustaw początkową maksymalną głębie pixela (i, j)

tzn. $d(i, j) = \text{nieskończoność (max wartość)}$

Ustaw początkowy kolor pixela (i, j) na kolor tła

$c(i, j) = \text{kolor tła}$

Dla każdego wielokąta wykonaj następujące kroki:

if pixel (i, j) znajduje się wewnątrz rzutu wielokąta na obraz
na obraz

{ znajdź odległość z pixela (i, j) od położonego
w wielokącie punktu $(x_{\text{bar}}, y_{\text{bar}}, z_{\text{bar}})$, którego rzutem jest
ten pixel

if $(z < d(i, j))$

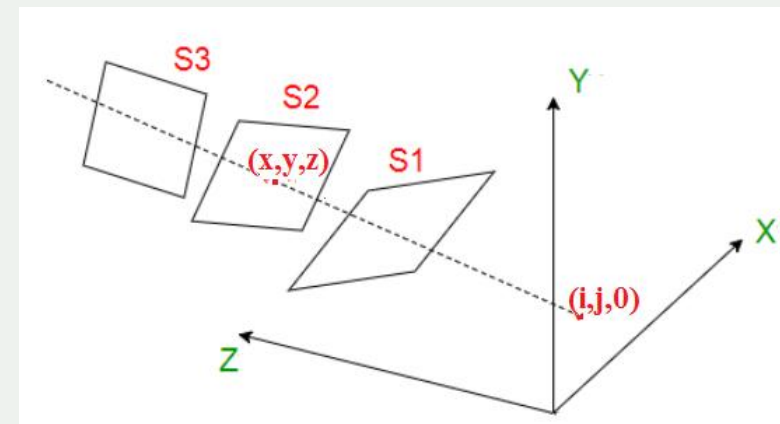
{

$d(i, j) = z;$

$c(i, j) = \text{kolor_wielokąta};$

}

}



Eliminacja elementów zasłoniętych

Algorytm z-bufora

Istotne punkty algorytmu

1. Jak sprawdzić czy spełnione jest

`if` pixel (i,j) znajduje się wewnątrz rzutu
wielokąta

2. Jak obliczyć

głębnię z pixela (i,j) , tzn. jego odległość od
położonego w wielokącie punktu $(x_{bar}, y_{bar}, z_{bar})$,
którego rzutem jest ten pixel

Eliminacja elementów zasłoniętych

Algorytm z-bufora

Istotne punkty algorytmu (Ad. 1)

Jak wyznaczyć, czy dany pixel (i,j) leży wewnątrz trójkąta utworzonego przez rzuty trzech narożników trójkąta 3D na obraz na płaszczyźnie OXY?

Dość skomplikowane w zapisie, ale proste obliczeniowo.

Narożniki trójkąta: $P_1 = (i_1, j_1)$, $P_2 = (i_2, j_2)$, $P_3 = (i_3, j_3)$

Wyznaczamy trzy proste: $A_2i + B_2j + C_2 = 0$, prosta P_1P_3

$A_1i + B_1j + C_1 = 0$, prosta P_2P_3

$A_3i + B_3j + C_3 = 0$, prosta P_1P_2

Pixel (i,j) leży wewnątrz (lub na krawędzi) trójkąta, jeśli spełnia poniższe warunki:

$$(A_1i_1 + B_1j_1 + C_1)(A_1i + B_1j + C_1) \geq 0$$

$$(A_2i_2 + B_2j_2 + C_2)(A_2i + B_2j + C_2) \geq 0$$

$$(A_3i_3 + B_3j_3 + C_3)(A_3i + B_3j + C_3) \geq 0$$

UWAGA: W praktyce problem ten jest często upraszczany (więcej na laboratorium).

Eliminacja elementów zasłoniętych

Algorytm z-bufora

Istotne punkty algorytmu (Ad. 2)

2. znajdź głębie z , tzn. odległość od punktu (i, j) w obrazie do punktu $(\bar{x}, \bar{y}, \bar{z})$ położonego w danym trójkącie i którego rzutem jest punkt (i, j)

Równanie płaszczyzny w przestrzeni wynosi:

$$Ax + By + Cz + D = 0$$

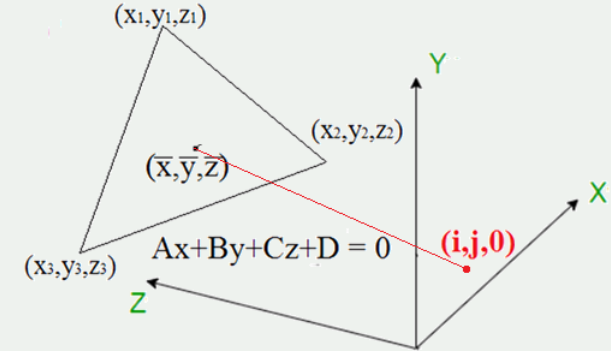
Płaszczyzna jest jednoznacznie zdefiniowana przez narożniki trójkąta

$$(x_1, y_1, z_1), (x_2, y_2, z_2), (x_3, y_3, z_3)$$

Wówczas powyższe równanie płaszczyzny jest równoważne równaniu:

$$\det \begin{bmatrix} x & y & z & 1 \\ x_1 & y_1 & z_1 & 1 \\ x_2 & y_2 & z_2 & 1 \\ x_3 & y_3 & z_3 & 1 \end{bmatrix} = 0$$

Łatwo z tego wyznacznika obliczyć parametry A , B , C i D równania.



Eliminacja elementów zasłoniętych

Algorytm z-bufora

Istotne punkty algorytmu (Ad. 2)

Trochę ogólnej matematyki:

Jak wyznaczyć punkt przecięcia płaszczyzny i prostej w 3D?

Prosta (równanie kierunkowe) $x = x_0 + l \cdot t, \quad y = y_0 + m \cdot t, \quad z = z_0 + n \cdot t$

Płaszczyzna $Ax + By + Cz + D = 0$

Punkt przecięcia $\bar{x} = x_0 - l \cdot \rho, \quad \bar{y} = y_0 - m \cdot \rho, \quad \bar{z} = z_0 - n \cdot \rho$

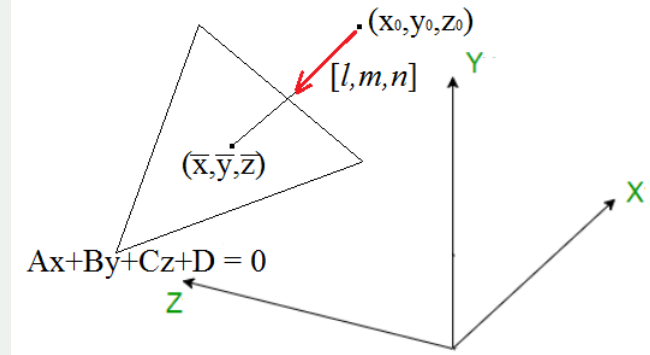
gdzie

$$\rho = \frac{Ax_0 + By_0 + Cz_0 + D}{Al + Bm + Cn}$$

Sytuacje szczególne

$Al + Bm + Cn = 0$ prosta jest równoległa do płaszczyzny

$\begin{cases} Al + Bm + Cn = 0 \\ Ax_0 + By_0 + Cz_0 + D = 0 \end{cases}$ prosta leży na płaszczyźnie



Eliminacja elementów zasłoniętych

Algorytm z-bufora

Istotne punkty algorytmu (Ad. 2)

Matematyka w przypadku algorytmu z-bufora:

Przy rzutowaniu równoległym $[l, m, n] = [0, 0, 1]$

Prosta (równanie kierunkowe) $x = i, \quad y = j, \quad z = 0 + 1 \cdot t$

Płaszczyzna $Ax + By + Cz + D = 0$

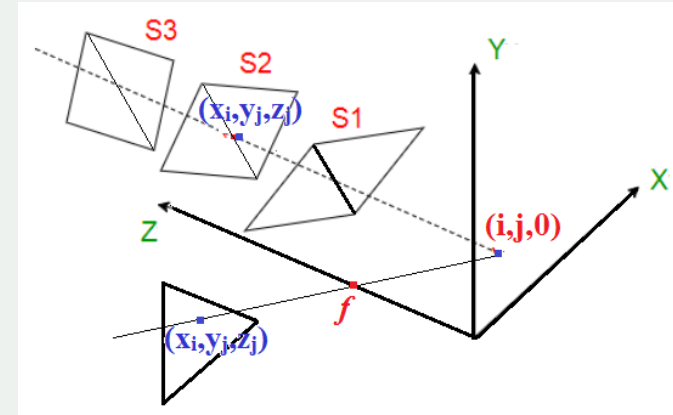
Punkt przecięcia

$$x_i = i, \quad x_j = j, \quad z_j = -\rho$$

gdzie

$$\rho = \frac{Ai + Bj + C \cdot 0 + D}{C} \quad \text{tzn.} \quad z_j = \frac{Ai + Bj + D}{-C}$$

$$\text{głębina} \quad z = \sqrt{(x_i - i)^2 + (y_j - j)^2 + z_j^2} = |z_j|$$

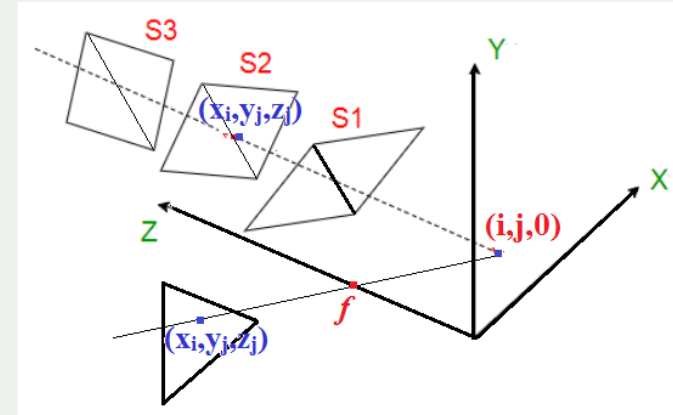


Eliminacja elementów zasłoniętych

Algorytm z-bufora

Istotne punkty algorytmu (Ad. 2)

Matematyka w przypadku algorytmu z-bufora:



Przy rzutowaniu perspektywicznym $[l, m, n] = [-i, -j, f]$

Prosta (równanie kierunkowe) $x = i - i \cdot t, \quad y = j - j \cdot t, \quad z = 0 + f \cdot t$

Płaszczyzna $Ax + By + Cz + D = 0$

Punkt przecięcia $x_i = i + i \cdot \rho, \quad x_j = j + j \cdot \rho, \quad z_j = -f \cdot \rho$

gdzie

$$\rho = \frac{Ai + Bj + C \cdot 0 + D}{-Ai - Bj + Cf}$$

$$\text{głębina} \quad z = \sqrt{(x_i - i)^2 + (y_j - j)^2 + z_j^2} = |\rho| \cdot \sqrt{i^2 + j^2 + f^2}$$

W ogóle nie potrzebujemy współrzędnych (x_j, y_j, z_j) punktu przecięcia!

Eliminacja elementów zasłoniętych

Algorytm z-bufora

Przykład

Dwie kopie trójkąta o wierzchołkach

$$(0,0,0), (1,0,0), (0,1,0)$$

(tzn. znajdującego się pierwotnie w płaszczyźnie obrazu) zostały pokolorowane na dwa różne kolory i każda kopia została przekształcona odpowiednio:

1. Obrócona w przestrzeni 3D z użyciem kątów RPY = $[30^\circ, 45^\circ, 10^\circ]$ a następnie przesunięta o wektor $[0.5, 0.3, 2]$.
2. Obrócona w przestrzeni 3D z użyciem kątów RPY = $[-30^\circ, -45^\circ, -10^\circ]$ a następnie przesunięta o wektor $[0.3, 0.5, 2]$.

Jak wyglądają rzuty obu trójkątów na płaszczyznę obrazu i i jaki obraz powinien powstać na płaszczyźnie obrazu (bo trójkąty te się częściowo zasłaniają i nawet przecinają)?

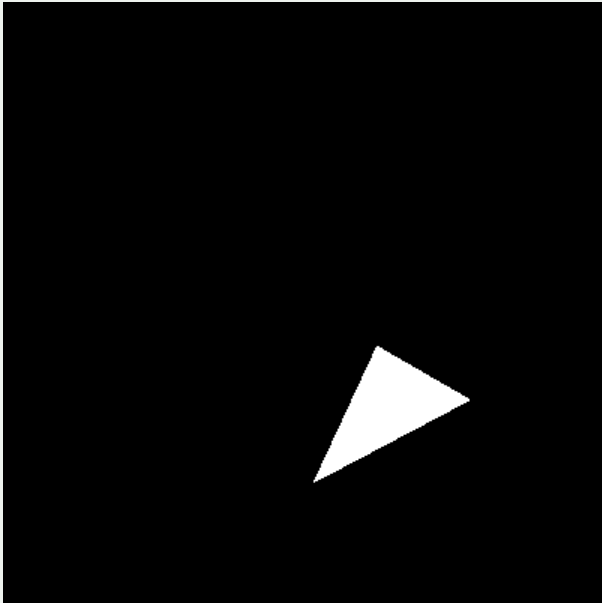
UWAGA: W przykładzie brakuje wielu detali. Czy rzut jest równoległy, czy perspektywiczny? Jaka jest ogniskowa kamery, jeśli rzut jest perspektywiczny? Jaka jest rozdzielczość obrazu cyfrowego i rozmiar pojedynczego pixela?

Eliminacja elementów zasłoniętych

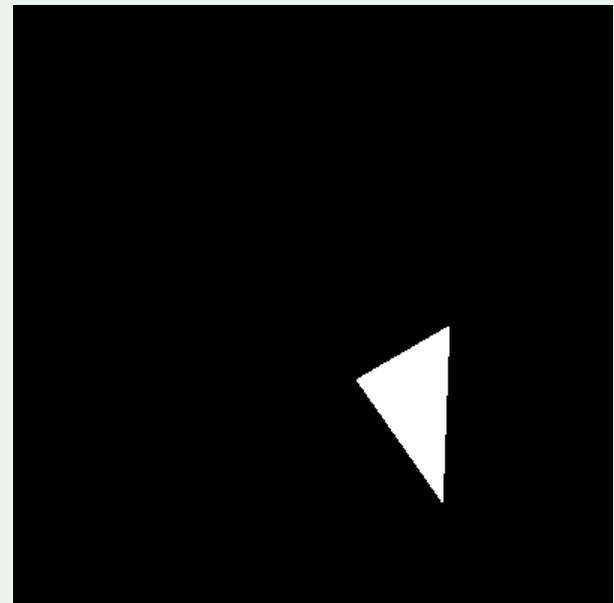
Algorytm z-bufora

Przykład

Rzut 1-go trójkąta na płaszczyznę obrazu



Rzut 2-go trójkąta na płaszczyznę obrazu

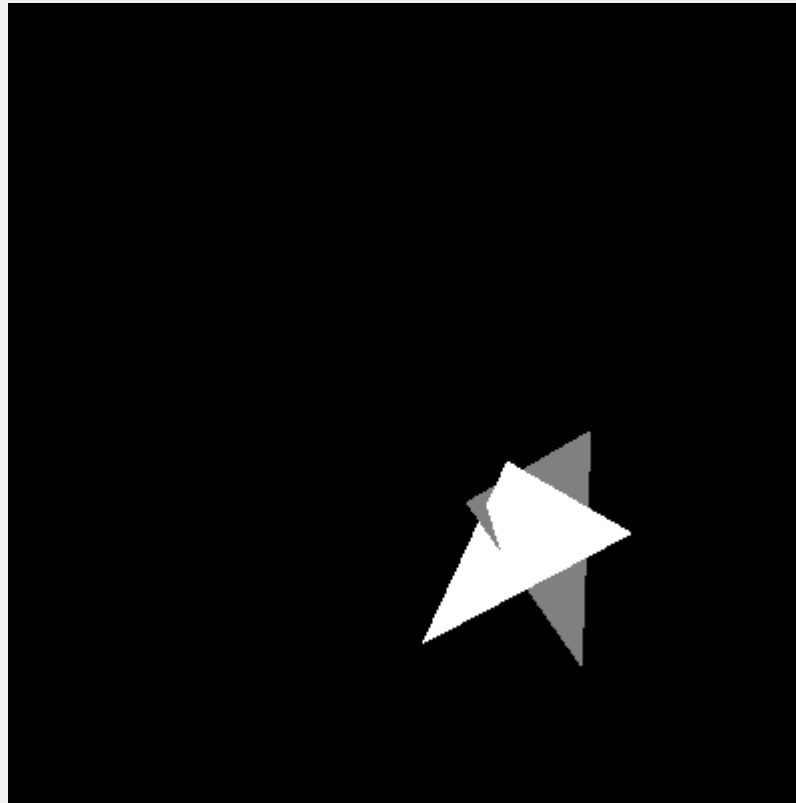


Eliminacja elementów zasłoniętych

Algorytm z-bufora

Przykład

Obraz wygenerowany z użyciem algorytmu z-bufora.



Eliminacja elementów zasłoniętych

Algorytm z-bufora (dodatkowe uwagi)

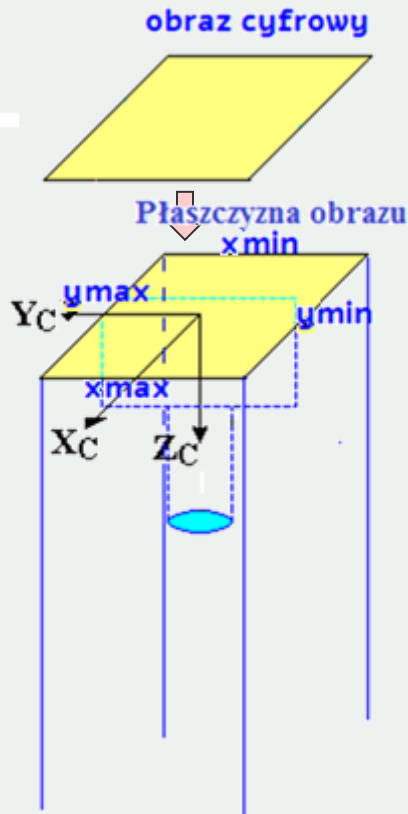
Złożoność obliczeniowa algorytmu z-bufora to $O(N \times K)$, gdzie jest N liczbą pixeli w obrazie, a K liczbą wielokątów.

Przy renderowaniu pojedynczego obrazu (cyfrowego) trójkąty, których na pewno nie będzie widać (tzn. w całości „wystają” poza rozdzielczość obrazu) mogą być ignorowane, co drastycznie zmniejsza złożoność obliczeń.

Algorytm „odfiltrowania” takich trójkątów jest trywialny w przypadku rzutu równoległego, a w przypadku rzutowania perspektywicznego tylko nieznacznie bardziej złożony (następne slajdy).

Eliminacja elementów zasłoniętych

Algorytm z-bufora (dodatkowe uwagi)



Przy rzucie równoległym, obraz może przedstawiać tylko te fragmenty sceny, które mieszczą się w prostopadłościanie zdefiniowanym przez:

$$\begin{cases} x_{\min} < x < x_{\max} \\ y_{\min} < y < y_{\max} \\ z > 0 \end{cases}$$

Pamiętajmy, że $x_{\min} = -x_{\max}$, $y_{\min} = -y_{\max}$, a ich konkretne wartości zależą od parametrów macierzy T_I^{pix} tzn. od rozdzielczości obrazu cyfrowego $m \times n$ i rozmiarów pixela $\Delta x \times \Delta y$ (patrz Wykład 3).

Oznacza to, że należy brać pod uwagę tylko te wielokąty (trójkąty) sceny, których co najmniej jeden narożnik spełnia powyższe nierówności.

Eliminacja elementów zasłoniętych

Algorytm z-bufora (dodatkowe uwagi)



Przy rzucie perspektywicznym, obraz może przedstawiać tylko te fragmenty sceny, które mieszczą się w stożku zdefirowanym przez poniższe nierówności (pamiętajmy, że $x_{\min} = -x_{\max}$ i $y_{\min} = -y_{\max}$):

$$z > f - \frac{f}{x_{\max}} x \quad \text{oraz} \quad z > f + \frac{f}{x_{\max}} x$$

$$z > f - \frac{f}{y_{\max}} y \quad \text{oraz} \quad z > f + \frac{f}{y_{\max}} y$$

Oznacza to, że należy brać pod uwagę tylko te wielokąty (trójkąty) sceny, których co najmniej jeden narożnik spełnia te nierówności.

Zauważmy, nierówności te pośrednio oznaczają, że zawsze $z > f$.